

Design Pattern

Prepared By: Team 03

Instructor
Md Samsuddoha

Assistant Professor
Department of Computer Science &
Engineering, University of Barishal

The Abstract Factory Pattern

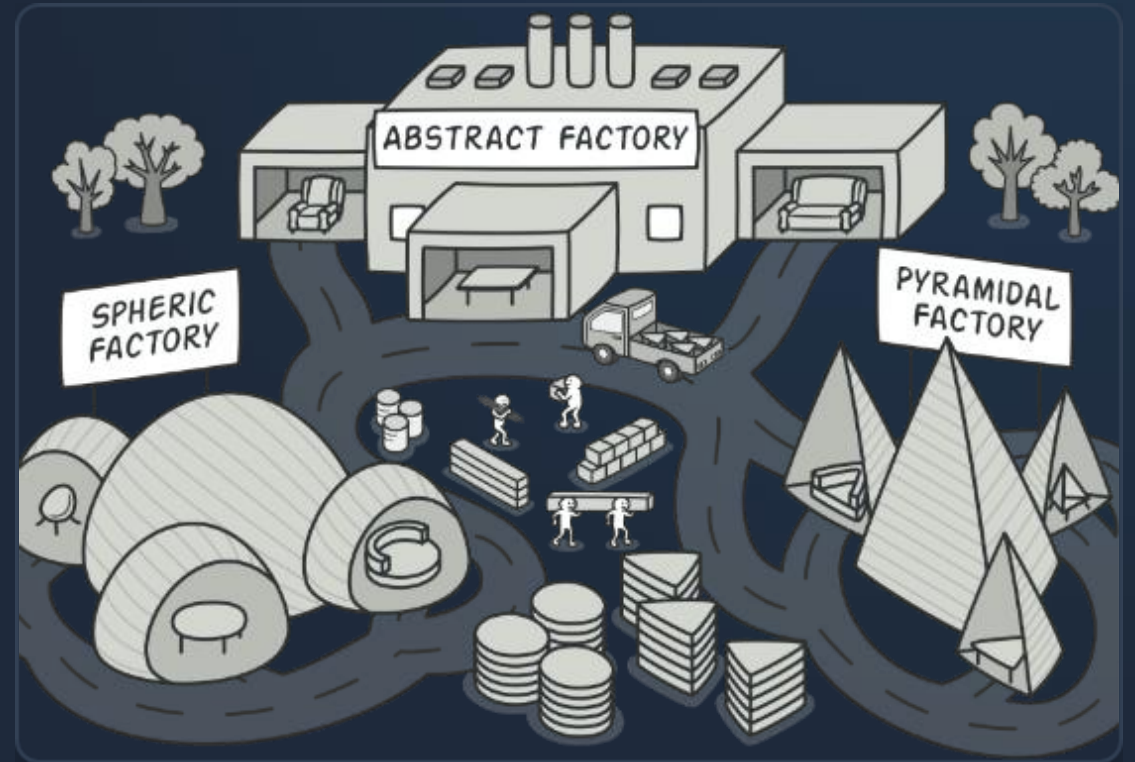
An industry-standard creational design pattern that facilitates the production of families of related or dependent objects without coupling client systems to concrete classes.

WHAT IS ABSTRACT FACTORY?

Creational Logic

Abstract Factory is a creational design pattern that lets you produce families of related objects without specifying their concrete classes.

It acts as a higher-level factory that creates other factories.



Intent of Abstract Factory



Consistency

Ensures that products from the same family are used together, maintaining visual and functional harmony.



Decoupling

Separates the creation of objects from their usage, allowing the client to work with any product variants.

Example : Furniture Families

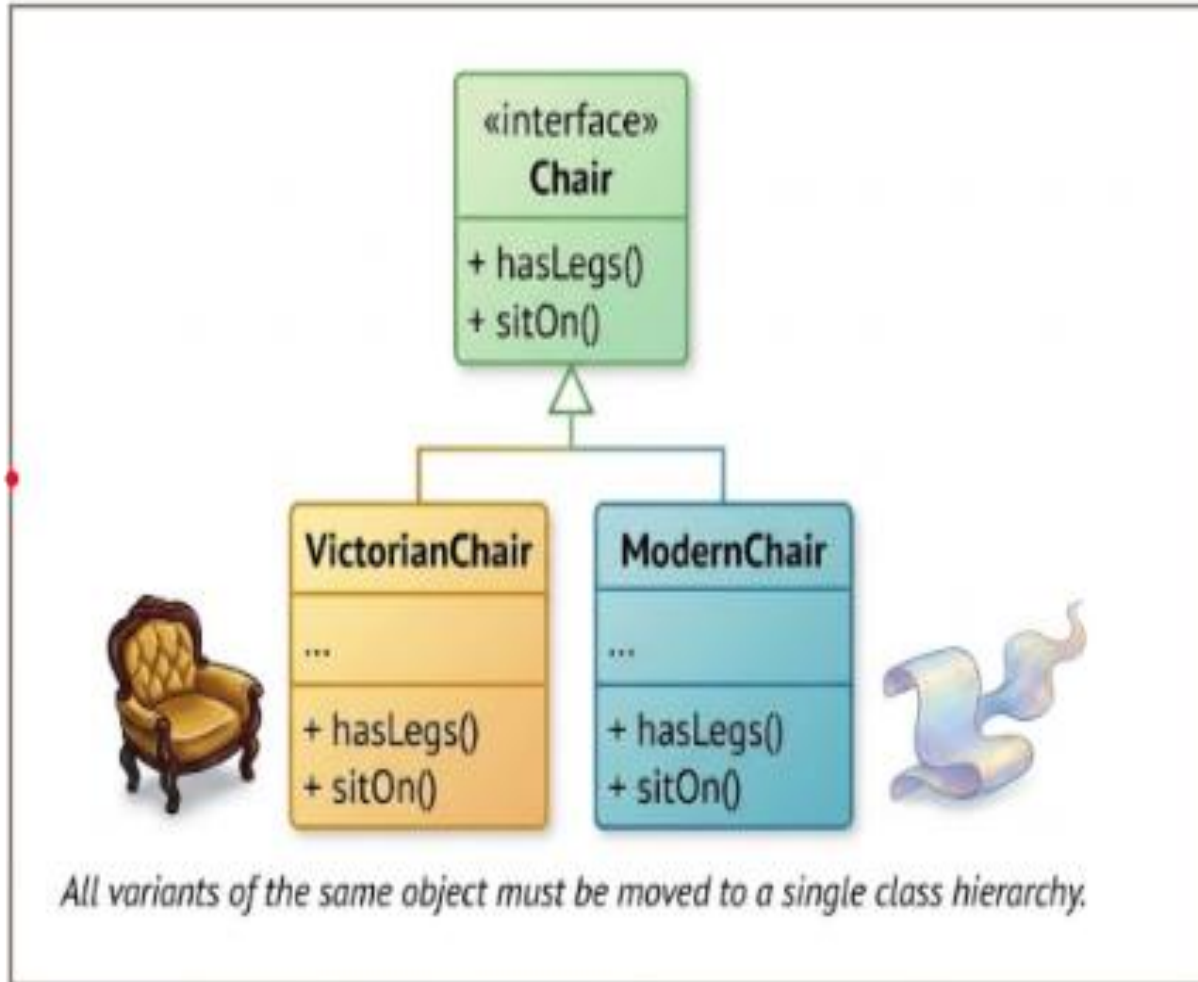


Mismatch Chaos

Imagine a furniture store where products like Chair, Sofa, and Coffee Table come in Modern, Victorian, and ArtDeco variants. Hard-coding specific classes leads to "Mismatching" — a Modern sofa in a Victorian room breaks the consistency of your system.

Solution (Abstract Factory)

Step I: Product Abstraction



Explicit Interfaces

The solution starts by declaring interfaces for each distinct product of the family (e.g., Chair, Sofa, Coffee Table).

All variants (Modern/Victorian) must follow these interfaces.

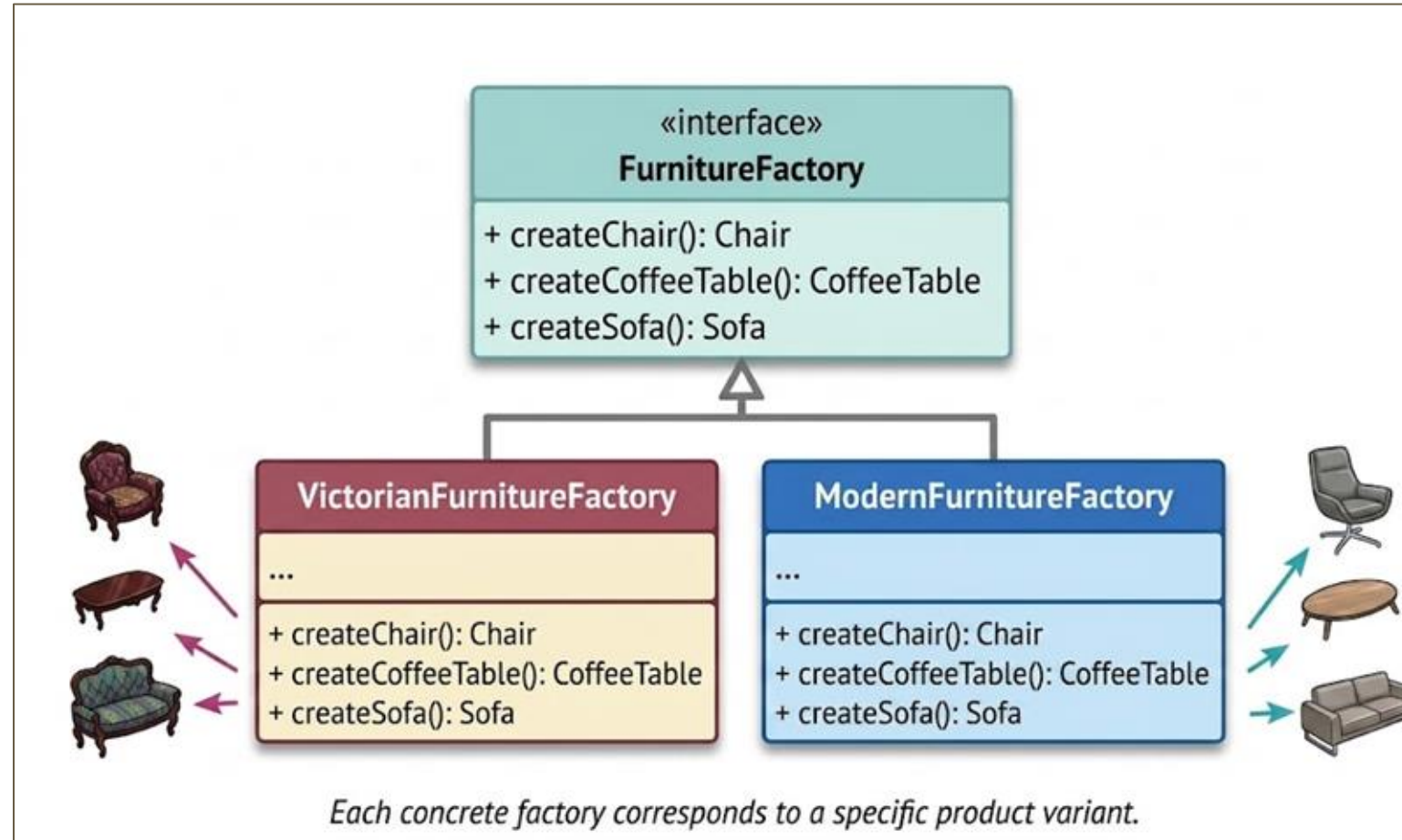
The client code works with the interface, not the concrete variant.

Step II:Factory Interface

The “Super” Factory

Declare the Abstract Factory- an interface with creating methods for every product in the family(e.g.,createChair(),createSofa()).

Each concrete factory(e.g., VictorianFactory) implements this interface and returns objects of a specific variant.



Result:Harmony



The Result Consistency

I. Client Doesn't Care

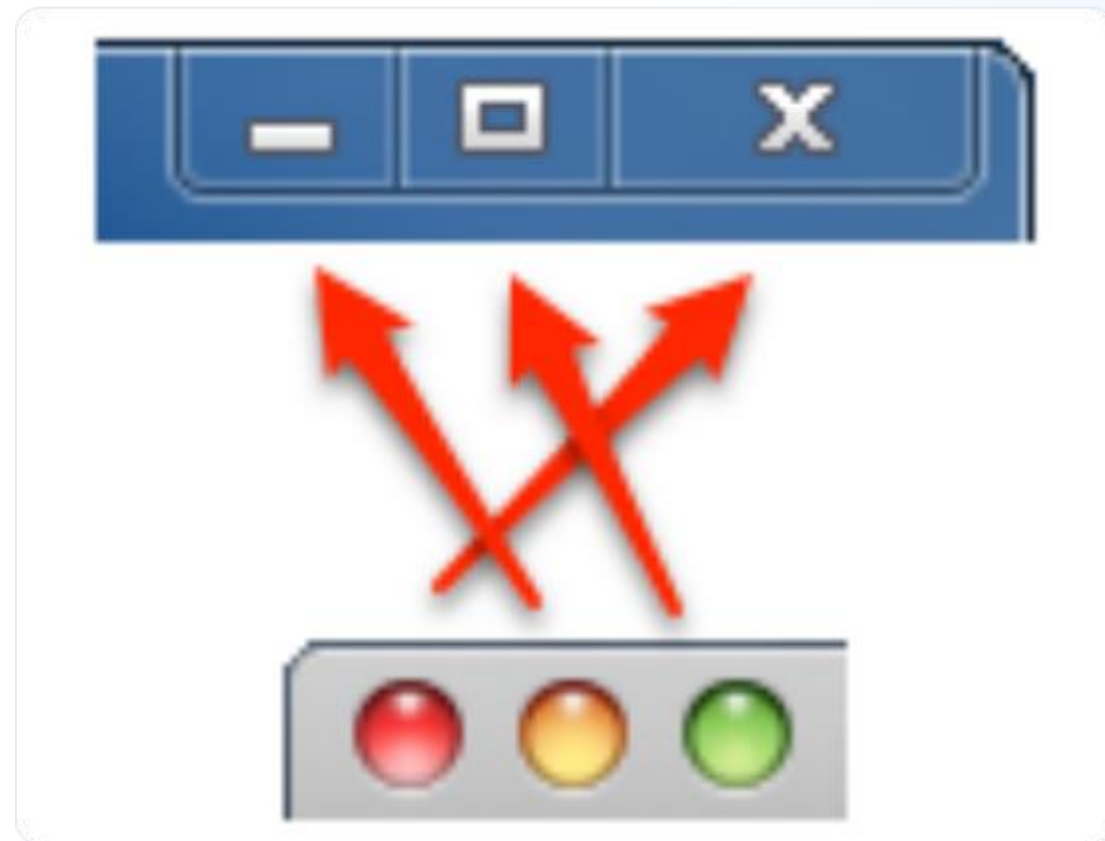
II.No More Mistake

Example II: Cross-Platform GUI

Look-and-Feel Inconsistency

In software, your app must look native on Windows and MacOS. Hard-coding `new WindowsButton()` forces you to rewrite code for Mac.

Worse, mixing a Mac button with a Windows window creates a jarring, unprofessional user experience.



2. The Problem (The “Family” Mismatch)

2. The Problem (The “Family” Mismatch)

We need our app to look native on each operating system.



Windows Requirements

- Buttons should look like Windows
- Checkboxes should look like Windows



macOS Requirements

- Buttons should look like Mac
- Checkboxes should look like Mac

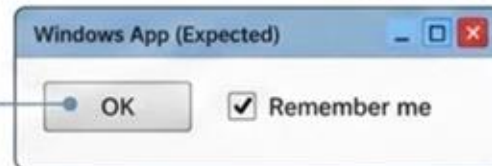


Bad Code (Family Mismatch!)

```
if (os.equals("Windows")) {  
    Button b = new WinButton();  
    Checkbox c = new MacCheckbox();  
    // BIG PROBLEM!  
}
```

We're on Windows, but created a macOS checkbox!
The UI will look broken.

Mismatched UI = Broken UI 🤔



≠



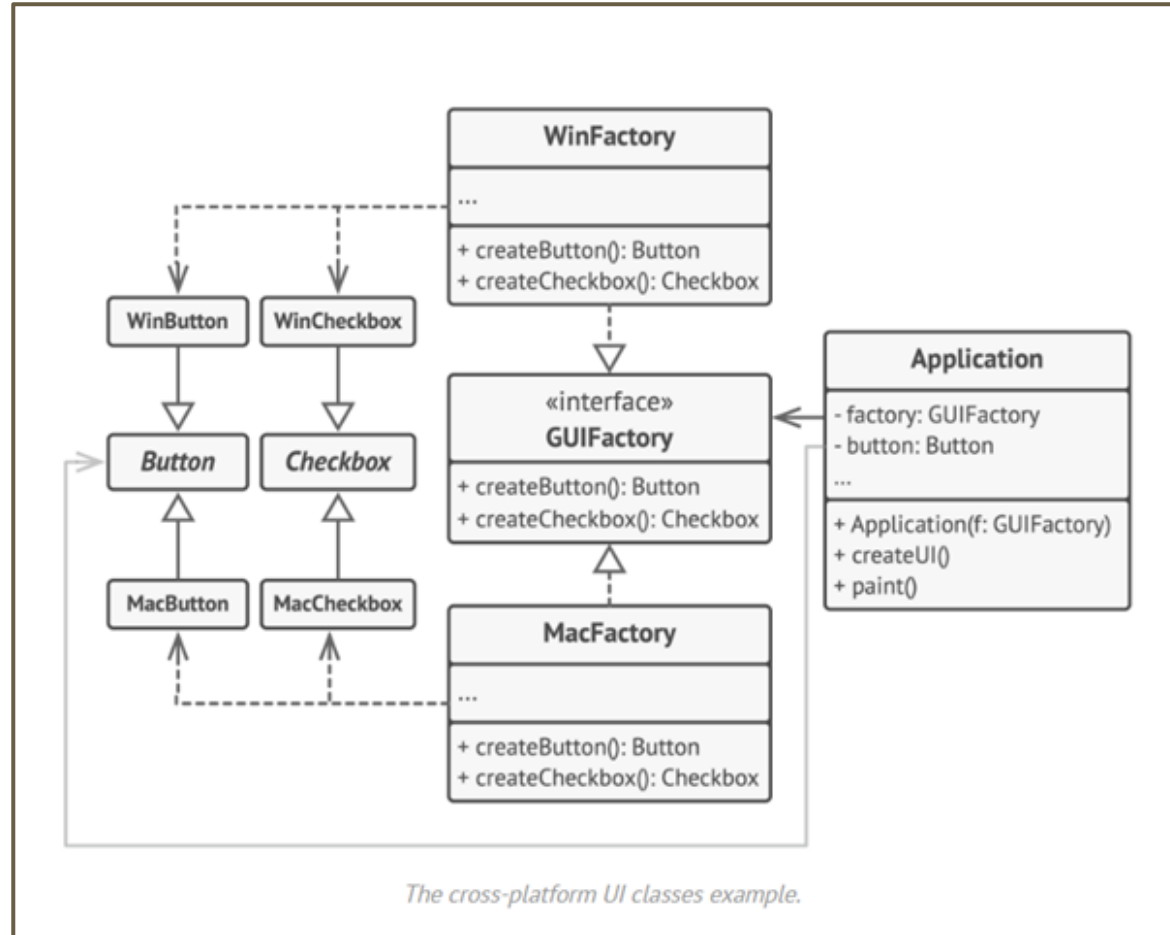
Windows button + Mac checkbox = Not a happy user!

1. The Context:
Desired Native
Platform Look

2. The Flaw: Concrete
Instantiation and
Tight Coupling

3. The Consequence:
Non-Deterministic
Mismatched UI Family

The Solution Overview



Solution - Abstract Products (Pseudocode)

❖ STEP 1: DECLARE ABSTRACT PRODUCT INTERFACES

- Each product in the family gets an interface.
- These are ABSTRACT PRODUCTS.

They declare WHAT each product can do, not HOW.

```
interface Button is  
    method paint()
```

```
interface Checkbox is  
    method paint()
```


Solution - Abstract Factory (Pseudocode)

❖ STEP 2: DECLARE ABSTRACT FACTORY INTERFACE

- One factory interface with creation methods for ALL products.
- This is the ABSTRACT FACTORY.

Note: Return types are interfaces (Button, Checkbox), not concrete classes.

```
interface GUIFactory is  
    method createButton(): Button  
    method createCheckbox(): Checkbox
```

SOLUTION – CONCRETE PRODUCTS (PSEUDOCODE)

✦ STEP 3: IMPLEMENT CONCRETE PRODUCTS

These are CONCRETE PRODUCTS. Each implements its interface.

WINDOWS FAMILY

```
class WinButton
    implements Button
    method paint()
        // Windows
        // style code
```

```
class WinCheckbox
    implements Checkbox
    method paint()
        // Windows
        // style code
```

MAC FAMILY

```
class MacButton
    implements Button
    method paint()
        // Mac style
        // code
```

```
class MacCheckbox
    implements Checkbox
    method paint()
        // Mac style
        // code
```

Solution - Concrete Factories (Pseudocode)

❖ STEP 4: IMPLEMENT CONCRETE FACTORIES

Each CONCRETE FACTORY creates
ONE family of products.

WinFactory → Only Windows
products

MacFactory → Only Mac
products

class WinFactory	class MacFactory
implements GUIFactory	implements GUIFactory
method createButton()	method createButton()
return new WinButton	return new MacButton
method createCheckbox()	method createCheckbox()
return new WinCheckbox	return new MacCheckbox

Java Implementation

```
// INTERFACES
```

```
public interface Button {  
    void paint();  
}
```

```
public interface Checkbox {  
    void paint();  
}
```

```
public interface GUIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}
```

```
// WINDOWS FAMILY
```

```
public class WinButton implements Button {  
    @Override  
    public void paint() {  
        System.out.println("Windows button");  
    }  
}
```

```
public class WinFactory implements GUIFactory {  
    @Override  
    public Button createButton() {  
        return new WinButton();  
    }  
  
    @Override  
    public Checkbox createCheckbox() {  
        return new WinCheckbox();  
    }  
}
```

```
// MAC FAMILY (similar structure)
```

```
public class MacButton implements Button {  
    @Override  
    public void paint() {  
        System.out.println("Mac button");  
    }  
}
```

```
public class MacFactory implements GUIFactory {  
    @Override  
    public Button createButton() {  
        return new MacButton();  
    }  
  
    @Override  
    public Checkbox createCheckbox() {  
        return new MacCheckbox();  
    }  
}
```

Client Code & Main

```
// CLIENT - Only uses interfaces!

public class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(GUIFactory factory) {
        this.button = factory.createButton();
        this.checkbox = factory.createCheckbox();
    }

    public void paint() {
        button.paint();
        checkbox.paint();
    }
}
```

```
// MAIN - Only place concrete classes appear

public class Demo {
    public static void main(String[] args) {
        GUIFactory factory;
        String os = System.getProperty("os.name");

        if (os.contains("Windows")) {
            factory = new WinFactory();
        } else {
            factory = new MacFactory();
        }

        Application app = new Application(factory);
        app.paint();
    }
}
```

OUTPUT on Windows:

- Windows button
- Windows checkbox

Output on macOS:

- Mac button
- Mac checkbox

Applicability



Related Families

When code must work with various families of related products.



Consistency

When you have a set of objects that **MUST** be used together.



Encapsulation

When you want to hide the construction details of products from clients.

How to Implement

- 1 Map out a matrix of product types vs. variants.
- 2 Declare abstract product interfaces for all types.
- 3 Declare the Abstract Factory interface with creation methods.
- 4 Implement concrete factory classes for each variant.

Pros & Cons

Advantages

- Compatibility guaranteed.
- Loosely coupled client code.
- Open/Closed Principle supported.

Disadvantages

- High code complexity.
- Too many new interfaces.
- Overkill for simple scenarios.

Relations with Patterns

Factory Method

Abstract Factory is often based on a set of Factory Methods.

Builder

AF creates products immediately; Builder constructs them in steps.

Singleton

Concrete Factories are often implemented as Singletons.

QUESTIONS?

Thank you for your attention!